This paper then involves extensive and comprehensive experiments conducted on 12 NLP datasets. These are meticulously designed to evaluate the performance (§4), usability, and underlying mechanisms, complete with in-depth analyses (§5), among other aspects. The key insights from our research are summarized as follows:

- **Performance.** We construct a new benchmark, named **KnowEdit**, and report the empirical results of cutting-edge knowledge editing approaches for LLMs, providing a fair comparison and illustrating their overall performance in the settings of knowledge insertion, modification, and erasure.
- **Usability.** We illustrate the impact of knowledge editing on general tasks and multi-task knowledge editing, which implies that contemporary knowledge editing methods are effective in executing factual updates with minimal disruptions to the model's cognitive capabilities and adaptability across diverse knowledge domains.
- **Mechanism.** We observe a pronounced focus on one or several columns within the value layer in edited LLMs. Furthermore, we find that the process of knowledge locating (e.g., causal analysis) tends to pinpoint only the areas related to the entity in question, rather than the entire factual context, suggesting that LLMs might be deriving answers either by recalling information memorized from their pretraining corpus or through a multi-step reasoning process. Additionally, we delve into the possibility that knowledge editing for LLMs could lead to unintended consequences, an aspect warranting careful consideration.

Finally, we delve into the multifaceted applications of knowledge editing, examining its potential from a variety of perspectives (§6), including efficient machine learning, AI-Generated Content (AIGC), trustworthy AI, and human-computer interaction (personalized agents). Additionally, our discussion extends to the broader impacts of knowledge editing techniques, specifically focusing on aspects such as energy consumption and interpretability (§7). This paper aims to serve as a catalyst for further research in the realm of LLMs, emphasizing efficiency and innovation. To support and encourage future research, we will make our tools, codes, data splits, and trained model checkpoints publicly accessible.

2 Background

2.1 Large Language Models

2.1.1 Transformers for LLM

The Transformer [77] model, a cornerstone in the design of modern state-of-the-art LLMs, represents a significant shift from previous sequence learning methods. The original Transformer model is introduced as an encoder-decoder framework, wherein both the encoder and decoder consist of a series of identical layers stacked upon each other. Each block within this architecture is equipped with a self-attention module and a fully connected feed-forward neural network. Uniquely, the blocks in the decoder also incorporate an additional cross-attention layer, positioned above the self-attention layer, which is designed to effectively capture and integrate information from the encoder.

Self-Attention Module (SelfAttn) The self-attention mechanism is a pivotal feature of the Transformer, allowing it to process sequences of data effectively. This module empowers each position within the encoder to attend to all positions in the preceding layer, thereby efficiently capturing contextual information embedded in the sequence. The mathematical representation of the self-attention mechanism is as follows:

$$H = \text{ATT}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \tag{1}$$

Feed-Forward Module (FFN) Following each attention layer in the Transformer is a fully connected Feed-Forward Neural network (FFN). This specific component of the architecture comprises two linear transformations, with a ReLU activation function intervening between them. The structure of the FFN can be succinctly described as follows:

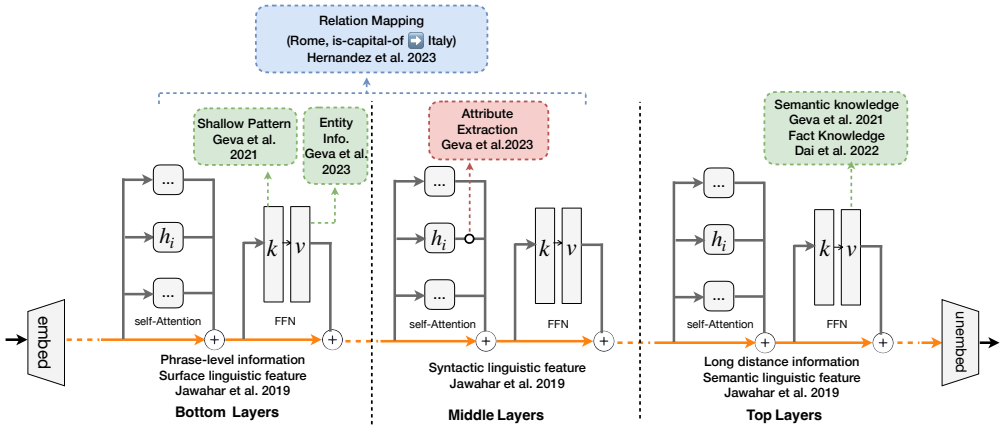


Figure 1: The mechanism of knowledge storage in LLMs. Here, we summarize the findings of current works, including: Jawahar et al. [78], Geva et al. [41], Dai et al. [39], Meng et al. [79], and Hernandez et al. [80].

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x} \cdot W_1 + b_1) \cdot W_2 + b_2, \tag{2}$$

Since its inception, the Transformer model has revolutionized the field of NLP. Its adaptable and efficient architecture has facilitated advancements in various NLP tasks, such as question-answering, text summarization, and machine translation systems. The model's influence extends beyond NLP, impacting other areas of machine learning and setting a new standard for building complex and effective neural network architectures.

2.1.2 Mechanism of Knowledge Storage in LLMs

The Transformer's remarkable performance is partly attributed to its ability to store a wealth of information within its parameters, encompassing linguistic [81], commonsense [82–84], arithmetic, and world knowledge [48, 85–87]. However, the exact manner in which this knowledge is organized within LLMs is still largely enigmatic. Current research efforts are dedicated to unraveling the mechanistic explanations of LLMs' behaviours [88–92], especially the complexities of knowledge storage in LLMs, with Figure 1 illustrating some of these research findings.

A key area of inquiry is pinpointing the specific location of knowledge within the model. Jawahar et al. [78] dissects the intricacies of the English language structure as comprehended by BERT [93]. Their findings reveal that BERT's phrasal representations capture phrase-level information predominantly in the lower layers, and encode an intricate hierarchy of linguistic elements in the intermediate layers. This hierarchy is characterized by surface features at the foundational level and syntactic features in the central layers, and culminates with semantic features at the uppermost level. Geva et al. [41] proposes that the FFN layers in a Transformer model function akin to key-value memories. They suggest that the FFN input operates as a query, with the first layer representing keys and the second layer corresponding to values. They find that human-interpretable shallow input patterns trigger each key neuron, and the corresponding value neurons store the next-token output probability. As a result, the final output of the FFN can be understood as the weighted sum of activated values. Furthermore, they demonstrate that value vectors often embody interpretable concepts and knowledge, which can be intensified or attenuated through specific manipulations [42]. Building on this, Dai et al. [39] introduces the concept of "Knowledge Neurons", suggesting that knowledge is localized within a small subset of FFN neurons in the uppermost layers of the language model. These neurons are identified through the analysis of integrated gradients across various prompts [94–96]. Similarly, Meng et al. [79] employs a method known as "causal tracing" to assess the indirect influences of hidden states or activations, revealing that factual knowledge predominantly resides in the early-layer FFNs of such models. Additionaly, Chen et al. [97] makes an intriguing finding that the language model contains language-independent neurons that express multilingual knowledge and degenerate neurons that convey redundant information by applying the integrated gradients method [94]. Concurrently, Zhao et al. [98] observes that LLMs appear to possess a specialized linguistic region

responsible for processing multiple languages. Gueta et al. [99] suggests that knowledge is a region in weight space for fine-tuned language models. They find that after finetuning a pretrained model on similar datasets, the resulting models are close to each other in weight space. Recent interests also revolve around dissecting the distinct functionalities of individual neurons within LLMs [100]. Yet, it is crucial to note that some researchers caution against overinterpreting these findings, emphasizing that models illustrate correlations rather than explicit mechanisms. For instance, Anonymous [101] argues that while MLP neurons may exhibit patterns interpretable through a linguistic lens, they do not necessarily "store" knowledge in a conventional sense, whether linguistic or factual.

Thus, the question of *how Transformer LLMs retrieve and utilize this stored knowledge* remains open, and some work has begun to unveil this mystery. Geva et al. [102] analyzes the information flow in the model and finds the self-attention model conducts attribute extraction during computing inspired by the circuit theory [103, 104]. Foote et al. [105] proposes Neuron to Graph (N2G), an innovative tool that automatically extracts a neuron's behavior from the dataset it was trained on and translates it into an interpretable graph. Further, Hernandez et al. [80] conceptualizes relational knowledge within Transformers as a linear affine function, mapping subjects to objects. As to other knowledge, Gurnee and Tegmark [36] discovers that LLMs learn linear representations of space and time across multiple scales and identify individual "space neurons" and "time neurons" that reliably encode spatial and temporal coordinates. However, it is imperative to acknowledge that these studies predominantly concentrate on the representation of individual knowledge facts. The broader challenge lies in comprehensively understanding how various strands of knowledge are intricately organized and interconnected within these complex models [106, 107].

2.2 Related Techniques

Parameter-efficient Fine-tuning Fine-tuning all parameters of LLMs can be computationally expensive. To enable efficient adaptation, parameter-efficient tuning (PET) [108, 109] techniques have been proposed to match full fine-tuning performance while only updating a minimal parameters. PET consists of three distinct paradigms: addition-based, specification-based, and reparameterization-based methods. In addition-based methods, extra trainable neural modules or parameters, which are not present in the original model or process, are introduced. A prime example of this is Adapter, as discussed in Houlsby et al. [110]. On the other hand, specification-based methods involve fine-tuning a select number of parameters, while keeping the majority of the model's parameters unchanged. A notable method in this category is LoRA, as detailed in Hu et al. [111].

By fine-tuning a small number of parameters, PET methods aim to maximize model performance while reducing required resources and tuning time. PET techniques hold promise since knowledge editing seeks to efficiently modify model behavior. However, PET is typically applied to enhance task performance rather than edit knowledge specifically. The efficacy of existing PET methods for knowledge editing remains largely unexplored. Investigating how to leverage PET for efficient and precise knowledge updates presents an interesting direction for future work.

Knowledge Augmentation for LLMs LLMs still face unknown questions, and many knowledge-augmented methods are proposed to help the model deal with this task [112–114]. The most popular way is the **retrieval-augmented** methods [115–117]. With the help of the retrieved knowledge or context that is related to the input, the model can give the desired output. The integration of the retrieved information includes both the input, intermediate, and output layers [118]. During the input phase, retrieved texts are concatenated with the original input text [119–121]. In some works, the retrieved components are latent and integrated into the intermediate layers of Transformers [122–124]. In the output phase, the distribution of tokens from the retrieved components and the LLMs are interpolated [125–128].

The knowledge-augmented method is a great solution for the missing or misinformation in LLMs but it still has some disadvantages. As a temporary solution, retrieval methods suffer from poor retrieval results and relatedness [129, 130]. The data retrieved often contains some noise, such as additional content that is irrelevant to a question but that may be relevant to a different question (i.e., not necessarily random noise) [131]. In these situations, the model fails to distinguish the knowledge that is necessary to answer the question, leading to spurious reasoning and degraded performance. Meanwhile, retrieval typically operates at a broader level of relevant passages without fine-grained control over precisely which information is modified within the model.